DevOps Report - Group C

Anders Juul Ingerslev (aing@itu.dk)
Asger Ramlau Jørgensen (asjo@itu.dk)
Johan Kristoffer Skoven Løye (jklo@itu.dk)
Malte Black Nielsen (mbln@itu.dk)

18th of May, 2026

Contents

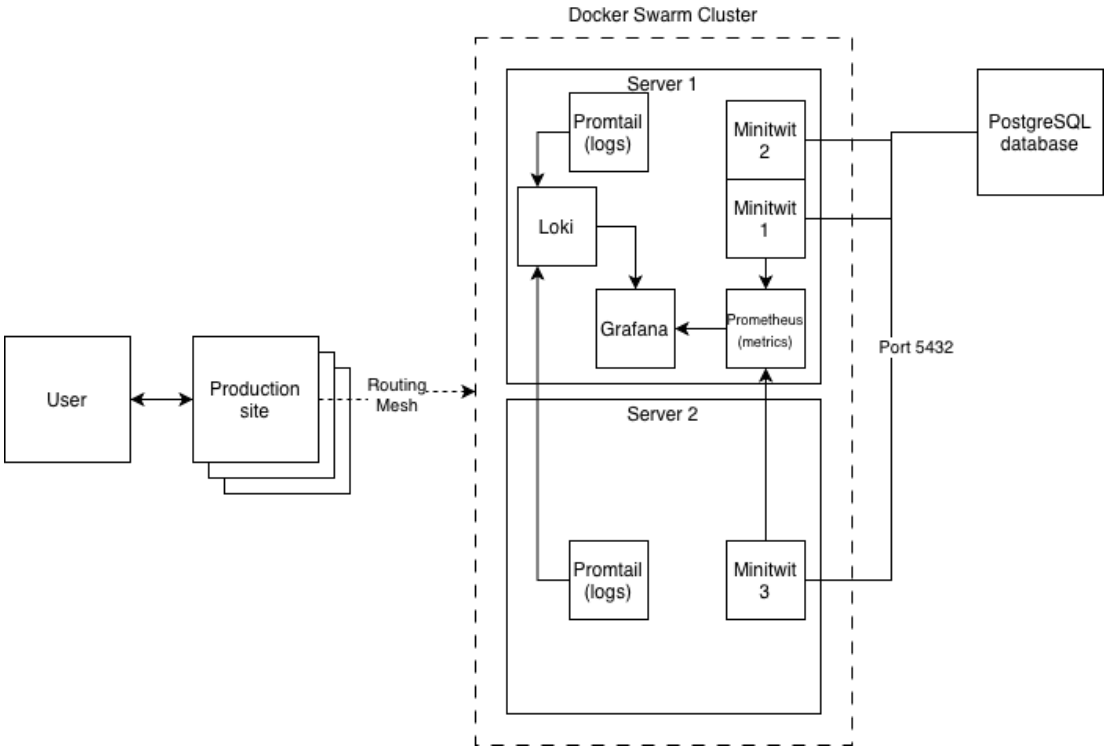

Figure 1: Architecture of the MiniTwit System

1 System's Perspective

1.1 Design & Architecture

1.1.1 Application

The minitwit application is a webapp twitter-like clone with simple functionality where users of the application can post tweets and follow each other.The application is written in Python with Jinja templates. The application has an API that contains the same functionality as the webapp.

1.1.2 Infrastructre

Infrastructure
Our system runs on three droplets hosted on DigitalOcean. Two swarm nodes form our swarm cluster. Minitwit-server 1 acts as a manager-worker and Minitwit-server 2 solely a worker. The last droplet is dedicated to hosting the sytem's Postgress database.

Besides managing container orchestration Mnitwit-server 1 is hosting 2 application replicas, a Promtail, Loki and Grafana replica. Minitwit-server 2 is hosting 1 application replica and a Promtail replica. Niginx is used as a reversed-proxy listening for incoming requests at port:80/443. Nginx forwards the given request and then the routing mesh of docker swarm forwards the request to an available replica. The routing mesh of the swarm ensures that each node in the cluster can handle any request regardless of which node is hosting the appropriate container.

1.1.3 Monitoring

Our system monitors the web application using Prometheus, Loki, Promtail and Grafana. Docker has been used for containerization of the sytem's services.

1.2 Dependencies

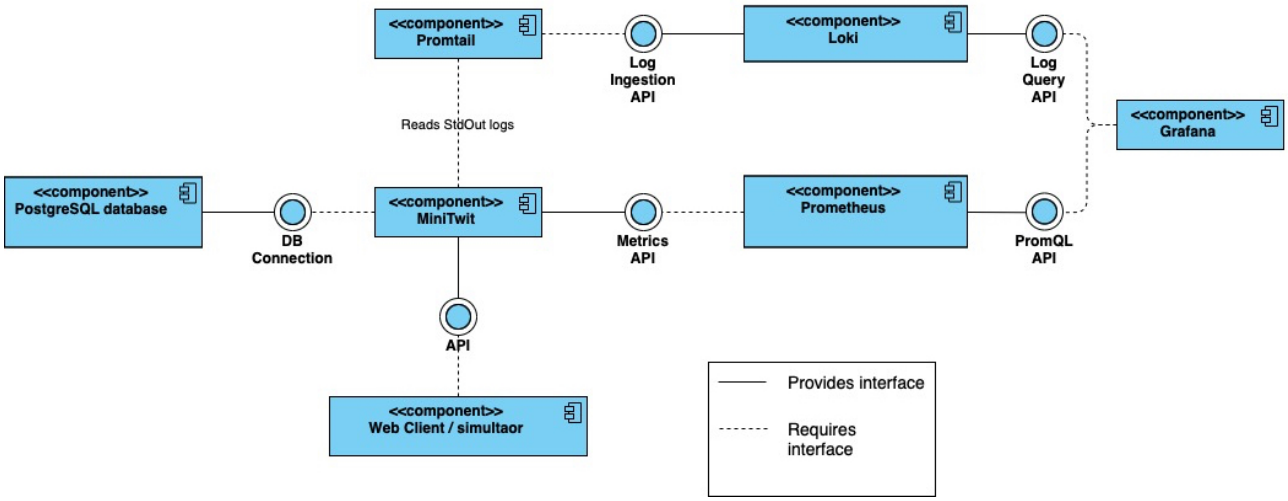

Figure 2: Component diagram of the MiniTwit system, showing provided interfaces and dependencies between the core application, database and monitoring services

1.2.1 Infrastructure

- **DigitalOcean Droplets**: Provides the hardware (CPU, Memory, Storage) to run a Linux OS on, hosting our services.
- **Docker**: Allows us to containerize our services.

- **Docker Swarm**: A part of Docker, allowing us to handle horizontal scaling and manage container health, update strategies and general management.
- **PostgreSQL**: A relational database management system, allowing us to write and read messages, log-in data and user-data concurrently.

1.2.2 Application

- **fastapi**: The asynchronous web framework handling routing, requests, and endpoints.
- **uvicorn**: The web server deployment dependency used to run the FastAPI application.
- **Jinja2**: The template engine used for server-side rendering of user timelines and layouts.
- **sqlmodel / SQLAlchemy**: The ORM used to abstract database interactions into Python code.
- **psycopg2-binary**: The PostgreSQL adapter required to handle high-concurrency network communication with our database container.
- **Werkzeug / itsdangerous** : Cryptographic utility relied upon for password hashing (**generate_password_hash**) and session tokens.

1.2.3 Monitoring

- **Prometheus**: A systems monitoring toolkit used to scrape and monitor time-series metrics.
- **Loki:** A log scraper that aggregates application text logs.
- **Promtail**: A service attached to all containers, which streams container stdout logs to Loki.
- **Grafana**: A data visualization and analytics web application, allowing us to query and monitor the output of the above and organize it in dashboards.

1.2.4 Pipeline

- **GitHub Actions**: Our CI/CD platform that automates our deployment, testing and building.
- **Docker Buildx and QEMU**: Compilation tools allowing us to create Docker Images that are agnostic to the OS they're being run on.
- **Black**: A code formatting tool allowing us to ensure an identical code style.

- **Hadolint**: An analysis tool that scans docker images for known vulnerabilities.
- **ShellCheck**: An analysis tool that scans shell files for bugs, syntax errors and vulnerabilities.

1.3 Current System State

Describe the current state of your systems, for example using results of static analysis and quality assessments.

2 Process' Perspective

2.1 Pipelines

Our pipeline is automated using GitHub Actions, triggered on every push and pull request to the main branch. We defined a four-stage process for it: Test, Analyze, Build and Deploy.

2.1.1 Test stage

The first stage is the testing stage. We use a docker compose file to spin up the application in a separate environment alongside the dependencies it needs, to ensure that the test-environment is as identical to the deployed environment as possible. The test container then runs all our defined tests via *pytest*. The container is ran with the command *"–exit-code-from test"*, ensuring that the exit code from the step is pulled from the tests. This means that if a test fails, the container exits with a non-zero code which stops the pipeline from proceeding further.

2.1.2 Analyze

The next step involves a group of static analysis tools. Unlike the previous stage, this stage does not execute the application or start containers. Instead, it reads the source code and files to ensure security and quality standards.

- **GitHub CodeQL**: we use CodeQL to perform an analysis of our codebase. CodeQL automatically identifies vulnerabilities such as SQL-injection points and insecure data handling, vulnerabilities that standard unit tests could miss. This step uses a list of vulnerabilities curated by GitHub's security team. This ensure that any push or pull request to main is analyzed for security before hitting the server.
- **Hadolint**: Hadolint is *"Smarter dockerfile linter that helps you build best practice Docker images"* [1]. It parses the image into an AST, and analyzes it for vulnerabilities.
- **Black**: A Python code formatter we've used to ensure code consistency across all our code, which serves to reduce "noise" in Git diffs, as well as ensure a consistent coding style across the codebase. While several linters / formatters exists for Python, we chose black as it seemed to be very well documented and highly regarded.
- **ShellCheck**: We used ShellCheck as a part of our pipeline to analyze any shell files we had, as none of us were very well versed in shell coding. ShellCheck ensured that small bugs and errors in any shell files, such as our *deploy.sh*, were picked up before being pushed to the server.

[1] https://github.com/hadolint/hadolint

2.1.3 Build

Once all test and analysis steps have finished, we proceed to the build stage. This stage is responsible for building the Docker images from the codebase, and checking vulnerabilities before they are then released to Dockerhub. This is done with Docker Buildx and QEMU. QEMU (*Quick Emulator*) allows us to make the resulting image agnostic to which OS it is ran on. This came as a result of the team having a mix of OS architectures, namely ARM (Mac) and x86 (Windows). While this just about doubles the pipeline duration, it was a necessity for the team.

Before the image is pushed to a public repository, we make use of *Docker Scout* to scan the image for vulnerabilities. Docker Scout creates a list of components and dependencies in the image and *"matches it against a continuously updated vulnerability database"*[2] It is configured with a *"fail-on: critical"* policy, ensuring that any part of the image that contains a vulnerability marked as "critical" stops the whole image from leaving the pipeline.

The image is then pushed to DockerHub with credentials safely secured in GitHub Secrets. After the push, a final security analysis is done with *Trivy*, a security scanner akin to Docker Scout. While this may very well be redundant, as Docker Scout and Trivy often overlap, we deemed it worth it for the potential that one might find an issue the other one did not.

2.1.4 Release and deployment

Once the image is pushed to DockerHub, the pipeline starts a SCP (Secure Copy Protocol) to synchronize a folder "*remote_files*" onto the production server. The pipeline then uses SSH to execute a script on the server, *deploy.sh*, which manages three tasks:

- 1: Pull the latest image image from DockerHub, the one pushed in this release.
- 2: Run *"docker stack deploy"*, creating a Docker Swarm to start a cluster of containers instead of just a single container.
- 3: Runs *"docker image prune -f"* to remove old images.

The Docker Swarm runs with 3 replicas, ensuring that up to 2 containers can go down at a time without causing an error for the end-user. The swarm is also running with *"order: start-first"* to minimize any downtime and ensure rolling updates. This means the swarm will spin up a new container with the new image, wait till it is healthy, and only then prune the old container before continuing on to the next container. It is configured with *"parallelism: 1"* to ensure only 1 of the 3 containers in the swarm is updated at a time.

[2] https://docs.docker.com/scout/

2.2 Monitoring & Logging

2.2.1 Monitoring

We have implemented a monitoring stack using Prometheus and Grafana. Both of these are containerized and deployed as part of our docker swarm.

Our Prometheus instance is set up to scrape time series metrics every 5 seconds from all three replicas of the application and our Grafana instance is set up to load pre defined dashboards that visualize these metrics. The flow can be seen in figure 4.

This is all to get a real-time overview of the systems health and performance. We maintain the following three dashboards:

- Health dashboard: Tells us the status (UP/DOWN) of the application and Prometheus itself.
- Performance dashboard: Visualizes various metrics of the application to give an indication of the performance. These include; total HTTP requests and rates, total HTTP failures and success rates, response time, etc.
- Database dashboard, showing current amount of unique users, and latest entries.

2.2.2 Logging

2.3 Security

We have implemented Nginx as a reverse proxy, which listens for traffic on port 80 (HTTP) & port 443 (HTTPS). HTTP requests are redirected to HTTPS. Any incoming traffic is forwarded to our minitwit application, which is running locally on the droplet. Nginx then fetches the response and sends it back to the client. This assures we only have a single point of access.
Certbot manages SSL certificates, ensuring all communication for end users is TLS encrypted.

We use Uncomplicated Firewall (UFW) to only expose certain ports.

2.4 Availability & Scaling

The application is available at `https://www.helgeandfriends.com/`. We used One.com to register the domain and add an A record to point to our public IP address (161.35.68.148).

As our traffic grew, we needed to scale our setup to accommodate for this increase. We did so by adjusting our Docker setup to swarm mode. Docker Swarm allows for multiple, concurrent services, meaning our application could

handle more requests. Additionally, it also helped our availability, as with multiple services, the application would still stay up and running if one were to fail.

3 Reflection Perspective

3.1 Evolution & Refactoring

We had some issues setting up TLS.....

3.2 Operation

3.2.1 Deployment of Monitoring Stack

When attempting to deploy our monitoring stack, we ran into several issues that prevented the update. Although the pipeline appeared to work, the server remained stuck on an older version because it was unable to get the new images.

Local Build Failures (Commit 6a222b2cf7c3fa155d9425e72b778eb4828a5caa)
We initially edited a legacy `docker-compose.yml` file in the root folder, which had no effect on production. After moving to the correct file in `remote_files/`, the deployment failed because the configuration incorrectly instructed the server to build the app from local source code, `remote_files/docker-compose.yml`:

```
prometheus:
build:
context: ../monitoring/prometheus
```

Since the server does not store the source code, it could not execute this build. As a result, the server could not generate the new image and continued running the version from 2 days ago. We resolved this in **Commit b20ee33b0e54661881a8e1e610f1260f77dea086** by instructing the server to pull pre-built images from Docker Hub instead `remote_files/docker-compose.yml`:

```
prometheus:
image: ${DOCKER_USERNAME}/prometheus_image:latest
```

Pipeline Misguidance (Commit 2667607971c13338e024e2e073c9636f1ffb544a)
Our GitHub pipeline reported success because it only verified that the SSH connection was established, not the outcome of the deployment script:

```
ssh $SSH_USER@$SSH_HOST "... && /minitwit/deploy.sh"
```

Evidence from the server (Appendix 5) confirmed this "false success." The system remained on a 2-day-old image (`cce05ed600a0`) even after we had pushed new changes. This confirmed that the script was failing while GitHub incorrectly signaled a green check mark. To fix this, we updated our pipeline to build the images in GitHub first, and then push them to Docker Hub.

This incident highlighted the danger of duplicate configuration files and silent pipeline failures. We have since removed the legacy root file and updated our workflow to ensure the pipeline indicates a failure if the deployment fails on the server.

3.2.2 Monitoring with Docker Swarms

After migrating to Docker Swarm our monitoring stack failed to scrape data and showed empty dashboards all of a sudden (see 6).

Initially we thought to fix this by hardcoding the server's IP address as a target for the data scraping, but we realized that in a swarm environment, containers are dynamic. They move between nodes and change internal IP's. A hardcoded target would only monitor a single instance, and would break if the Swarm moved the container to a different address.

We resolved this in **Commit 5b452bd9cd28dd3b8b6a78b44c4141168097d986** by switching to DNS service discovery. Instead of a fixed IP, we used the `tasks.` prefix, which tells Prometheus to query Docker's internal DNS for the IP's of all three running replicas `monitoring/prometheus/prometheus.yml`:

```
dns_sd_configs:

names: ['tasks.minitwit_minitwit']
type: 'A'
port: 5001
```

This allowed Prometheus to automatically detect and scrape all three containers, as confirmed by the three active endpoints in Figure 7.

3.2.3 API for simulator

When we initially set up the API for the simulator, we ran into an issue with correctly defining the endpoints. None of us had before used FastAPI, so we were not completely aware of the ordering of the defined endpoints. This lead to about a week of missed data from the simulator. This was fixed in **commit 20898c14851290e95260c65ecea6592bc5829c23**. Because of this, we saw a lot of errors for a short period of time, where the simulator attempted to interact with MiniTwit via users that were not stored in our database. We discussed seeing if we could obtain a snapshot of another teams database, but decided that it was unnecessary as new users would, over time, be created by the simulator.

3.3 Maintenance

3.4 Workflow

Initially, we struggled to maintain an overview of weekly tasks and distribute them among the team. This led to instances where multiple members unknowingly worked on the same task, resulting in duplicated work.

To help with this we adopted GitHub issues, where we could add all the main tasks of the week with descriptive sub tasks for each of them. This workflow

gave us a clear overview of our progress and allowed us to link our planning directly to working on the task by creating branches and Pull Requests directly from specific issues.

4 Diagrams

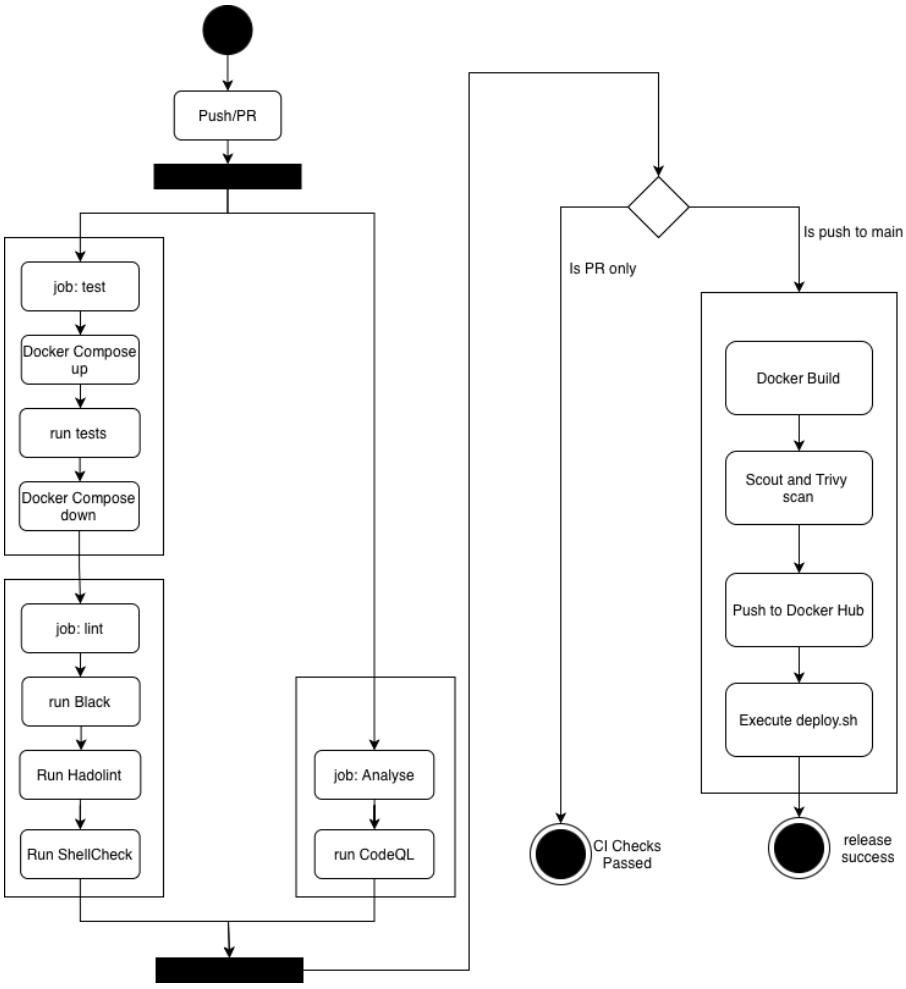

Figure 3: UML activity diagram of CI/CD

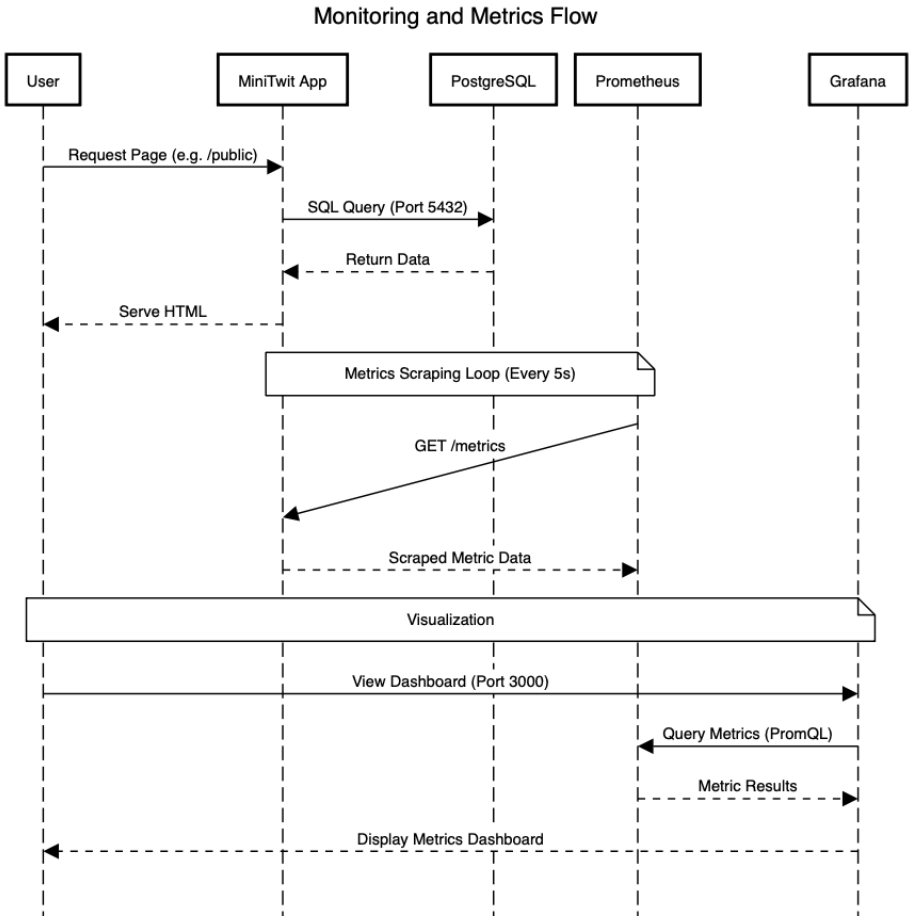

Figure 4: UML sequence diagram showing monitoring flow

5 Appendix

```
root@minitwit-server:~# docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED      STATUS      PORTS                                       NAMES
47d26fcf157c   cce05ed600a0   "uvicorn minitwit:ap…"   2 days ago   Up 2 days   0.0.0.0:80->5001/tcp, [::]:80->5001/tcp   minitwit-prod
root@minitwit-server:~# docker images
REPOSITORY              TAG       IMAGE ID       CREATED          SIZE
jskoven/minitwitimage   latest    0de17d701c40   37 minutes ago   417MB
jskoven/minitwitimage   <none>    cce05ed600a0   2 days ago       417MB
root@minitwit-server:~#
```

Figure 5: docker ps command

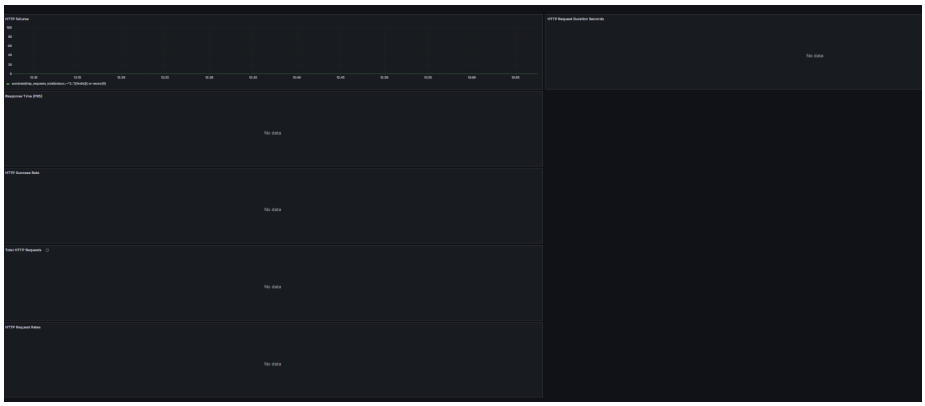

Figure 6: Grafana dashboards showing no data

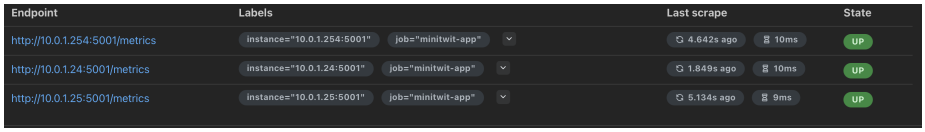

Figure 7: Prometheus health board